COMP1350 Group Project

Session: **2** Year: **2025**

Group ID: **z13-D**

STUDENT ID	STUDENT FULL NAME (as shown on iLearn)
49152467	Lea Macalindong
48953202	Ansh Verma
49021818	Shah Noor Mostafa Bhuiyan(Group Leader)
49176749	Biola Ngang

Scenario

background:

Alex has always been interested in food. They want to make a personal app that anyone can download and use for their own purposes - not to be shared. This app (Foodalicious) would allow any person to start building up their own bank of recipes for their own personal collection.

This isn't just an ordinary "here is a meal" with instructions and a nice photo or two… no. Alex has a thing for breaking things down and being able to filter and search by different properties. So, Alex wants to be able to catalogue many different things about each recipe.

Your mission is to take the scenario business needs (below) and do a conceptual design of the system, transform that completed conceptual design into a logical design, and then convert it to a physical design targeting MySQL-compatible data types.

- tables (entities) - relationships - attributes

scenario & business needs:

In Alex's App, a recipe will need to be broken down into many different aspects. Each recipe will have a number that can be used by the app to uniquely identify the recipe. The recipe should also have a name, a date that it was entered into the app, and a difficulty rating from 1 to 5 for how difficult it is to make.

Alex would like to be able to identify any ingredient used, so storing the details of each ingredient is also important for this app. The ingredient should have a name, and also the ability to store 'other names', and be uniquely identified by an ingredient number in the app. The ingredient will also need to be classified by an ingredient type (e.g. a spice, meat, fungus, …), which has its own unique identifier and a name.

Alex likes to catalogue everything in detail, therefore wants to capture each kitchen tool needed in each recipe. So, the kitchen tool also has a unique identifier and a name. Other similar things that will need to be cataloged with unique identifiers are 'dietary category' (with example names like vegetarian, vegan, halal, …), 'meal type' (with example type names like breakfast, lunch, dinner, snack, …), 'cuisine' (with example cuisine names like Italian, Japanese, Indian, …), unit of measurement (with example measure names like cups, teaspoons, mL, grams, …), ingredient form (with names of forms like sliced, diced, shredded, melted, …), and a cooking method (with some example names like frying, baking, mixing, kneading, crushing, …).

Each recipe has many process steps to be followed in order. Each process step is identified by the step number (e.g. step 1, step 2, step 3) for a particular recipe. The

process step also has a description and may store an image (as a filename) that refers to the location of the image in the app's file storage area.

Each process step may relate to a cooking method and may require the use of some kitchen tools during the process. That kitchen tool usage for the process step also needs to have a description.

Each recipe usually has a list of ingredients, but we will call each of them an ingredient item. This would associate each ingredient used in a recipe with the ingredient form and the unit of measurement. The ingredient item would store the amount needed of that unit of measurement for the recipe.

Alex has the unusual observation that it is possible for an ingredient to be made up of multiple ingredients, so the relationship between ingredients should also be captured (if any exist). For example, aioli can be made using olive oil, garlic, lemon juice, and salt.

One final thing that Alex wants to achieve is to assemble a meal plan for hosting dinners. The meal plan can be uniquely identified and have a name (like Bob's birthday), date, start time, and duration in hours, and will plan out meals for the event. The meal plan would have at least one starter meal (each as a recipe), at least one main meal (each as a recipe), and optionally any number of dessert meals (each as a recipe) if so desired.

Task 1: Conceptual Design

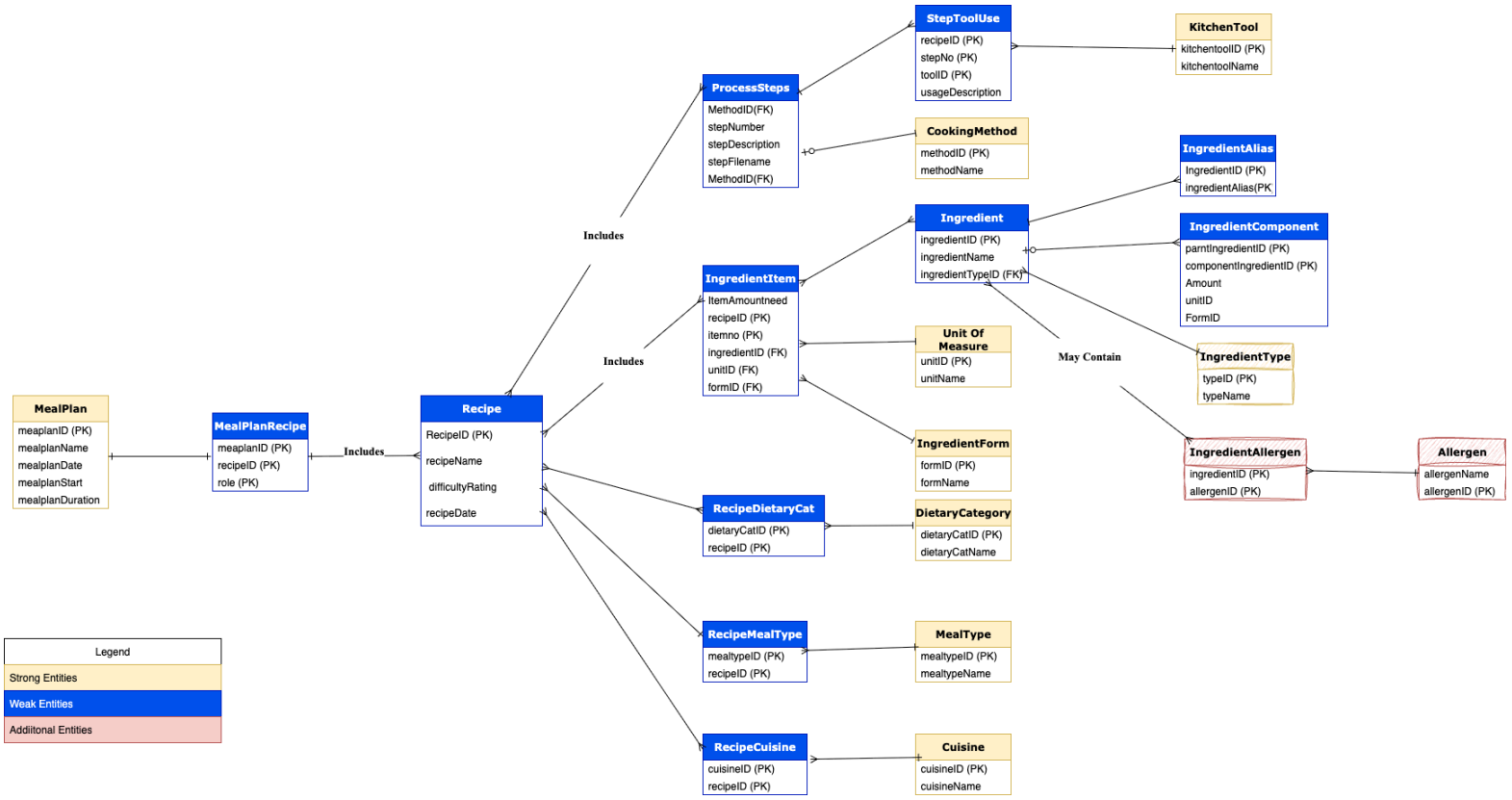
business rules	
1.	Each Recipe is uniquely identified by its RecipeID and should have Name, EnteredDate along with difficulty rating 1 to 5.
2.	Each recipe may have one or more process steps, however, a process step must only belong to one recipe and is identified by **(RecipeID, StepNo)**, and has **Description** and optional **ImageFilename**.
3.	A recipe may have zero or more ingredient items Each IngredientItem must specify the Ingredient, Amount, Unit and Ingredient Form.
4.	Each process step may be linked to zero or one cooking method And **each ProcessStep may use zero or more KitchenTools;** each usage can include a **usage description**
5.	**Ingredient** is uniquely identified by **IngredientID**, has **Name**, and **belongs to exactly one IngredientType**
6.	Each ingredient type has its own name and unique ID number in order to identify it and may classify many ingredients
7.	A recipe may consist of zero or more Dietary categories and each Dietary category may apply to many recipes
8.	Each recipe must be linked to at least one meal type; a meal type may apply to many recipes
9.	A Recipe can be linked with zero or more Cuisines and each Cuisine can be applied with many Recipes.
10.	An Ingredient may be composed of zero or or more other ingredients and each component used must be specified with its Amount, Unit, and a Form.
11.	**Each IngredientItem** specifies exactly **one Ingredient**, **one UnitOfMeasure**, and **one IngredientForm**, and stores an **Amount (>0)**.
12.	A MealPlan is uniquely identified by its MealPlanID along with Name, Date, StartTime, DurationHours.It must include at least one Starter and one Main recipe, and may include zero or more Dessert recipes. **An Ingredient may have zero or more Aliases** (“other names”), each tied to that particular Ingredient.
13.	**An Ingredient may have zero or more Aliases** (“other names”), each tied to that particular Ingredient.

14.	**KitchenTool (ToolID)**, **Cuisine (CuisineID)**, **MealType (MealTypeID)**, **DietaryCategory (DietCatID)**, **UnitOfMeasure (UnitID)**, **IngredientForm (FormID)**, and **CookingMethod (MethodID)** each have a unique identifier and **Name**.

additional business rule
Each Ingredient may be linked to zero or more Allergens (e.g., peanut, gluten). Each Allergen may link to many Ingredients.

list of assumptions	
1.	The recipe difficulty is set between 1 and 5 to prevent people from entering 0, 10 or easy by mistake
2.	In order to maintain a tidy database and prevent name repetition in the ingredients table, the ingredients are stored in a separate table with one different name per row.
3.	Units are stored as written to prevent auto conversion such as “1 cup” will stay “1 cup” instead of converting to “250ml”
4.	If an image is necessary for a particular step, the database only stores the file name/path and not the image itself while the app manages the files.
5.	A step may or may not have a cooking method associated with it as certain steps do not require a method. (e.g “let it rest for 5 minutes”)
6.	An ingredient cannot directly or indirectly include itself to avoid repetition or redundancy. (e.g “aioli is made from aioli”)
7.	A meal plan must have at least one starter,one main before its completed but we can keep the file incomplete while creating it before finalisation.
8.	A recipe can be made without the list of steps or in ingredients while it's being created but can't be published without them.
9.	Cuisine, IngredientType,Unit etc must all be unique as they can’t have any duplicates to avoid confusion.
10.	To keep itself simple, the database adjusts the date and time according to the user’s location minimising user interference.
11.	“Attributes amount, unit id, and form id in IngredientComponent can be optional, since a composite ingredient may not always specify quantity, unit, or form for each component.”

ER Model


MealPlan
meaplanID (PK)
mealplanName
mealplanDate
mealplanStart
mealplanDuration
MealPlanRecipe
meaplanID (PK)
recipeID (PK)
role (PK)
Includes
Recipe
RecipeID (PK)
recipeName
difficultyRating
recipeDate
Includes
Includes
ProcessSteps
MethodID(FK)
stepNumber
stepDescription
stepFilename
MethodID(FK)
StepToolUse
recipeID (PK)
stepNo (PK)
toolID (PK)
usageDescription
KitchenTool
kitchentoolID (PK)
kitchentoolName
CookingMethod
methodID (PK)
methodName
IngredientItem
ItemAmountneed
recipeID (PK)
itemno (PK)
ingredientID (FK)
unitID (FK)
formID (FK)
Ingredient
ingredientID (PK)
ingredientName
ingredientTypeID (FK)
IngredientAlias
IngredientID (PK)
ingredientAlias(PK
IngredientComponent
parntIngredientID (PK)
componentIngredientID (PK)
Amount
unitID
FormID
Unit Of Measure
unitID (PK)
unitName
May Contain
IngredientType
typeID (PK)
typeName
IngredientForm
formID (PK)
formName
IngredientAllergen
ingredientID (PK)
allergenID (PK)
Allergen
allergenName
allergenID (PK)
RecipeDietaryCat
dietaryCatID (PK)
recipeID (PK)
DietaryCategory
dietaryCatID (PK)
dietaryCatName
RecipeMealType
mealtypeID (PK)
recipeID (PK)
MealType
mealtypeID (PK)
mealtypeName
RecipeCuisine
cuisineID (PK)
recipeID (PK)
Cuisine
cuisineID (PK)
cuisineName
Legend
Strong Entities
Weak Entities
Addiitonal Entities

Task 2: Logical Design

Logical Transformation:

Step 1: Strong Entities → Relations

We have converted the main entities into tables, each with its own primary key:

- Recipe (recipeID PK, recipeName, difficultyRating, recipeDate)
- Ingredient (ingredientID PK, ingredientName, ingredientTypeID FK)
- IngredientType (typeID PK, typeName)
- KitchenTool (kitchenToolID PK, kitchenToolName)
- CookingMethod (methodID PK, methodName)
- Cuisine (cuisineID PK, cuisineName)
- DietaryCategory (dietaryCatID PK, dietaryCatName)
- MealType (mealTypeID PK, mealTypeName)
- UnitOfMeasure (unitID PK, unitName)
- IngredientForm (formID PK, formName)
- MealPlan (mealPlanID PK, mealPlanName, mealPlanDate, mealPlanStart, mealPlanDuration)
- Allergen (allergenID PK, allergenName)

Step 2: Weak Entities → Relations

Weak entities usually depend on other tables for their meaning.

- IngredientItem (recipeID PK+FK, itemNo PK, ingredientID FK, unitID FK, formID FK, itemAmountNeeded)

- ProcessSteps (recipeID PK+FK, stepNumber PK, stepDescription, stepFilename, methodID FK)

- StepToolUse (recipeID PK+FK, stepNo PK+FK, toolID PK+FK, usageDescription)

Step 3: One-to-One Relationships

There are no true 1:1 relationships in this scenario.
So not applicable.

Step 4: One-to-Many Relationships

Ingredient (ingredientID (PK), ingredientName, ingredientTypeID (FK))

- ingredientTypeID is a foreign key referencing IngredientType(typeID).

ProcessStep (recipeID (PK)(FK), stepNo (PK), stepDescription, stepFilename, methodID (FK))

- recipeID is a foreign key referencing Recipe(recipeID).
- methodID is an optional foreign key referencing CookingMethod(methodID).

Ingredient (ingredientID (PK), ingredientName, ingredientTypeID (FK))
IngredientItem (recipeID (PK)(FK), itemNo (PK), ingredientID (FK), unitID (FK), formID (FK), amount)

- recipeID links IngredientItem to Recipe.
- ingredientID links IngredientItem to Ingredient.
- unitID links IngredientItem to UnitOfMeasure.
- formID links IngredientItem to IngredientForm.

Step 5: Many-to-Many Relationships

We have solved M:N by creating linking (associative) tables:

- RecipeCuisine (recipeID PK+FK, cuisineID PK+FK) → allows recipes to have many cuisines.

- RecipeDietaryCat (recipeID PK+FK, dietaryCatID PK+FK) → allows recipes to have many dietary categories.

- RecipeMealType (recipeID PK+FK, mealTypeID PK+FK) → allows recipes to have many meal types.

- MealPlanRecipe (mealPlanID PK+FK, recipeID PK+FK, role PK) → allows a meal plan to include many recipes with a role like Starter, Main, or Dessert.

- IngredientAllergen (ingredientID PK+FK, allergenID PK+FK) → links ingredients with allergens.

Step 6: Multi-Valued Attributes

Multi-valued attributes are stored in their own tables such as:

- IngredientAlias (ingredientID PK+FK, ingredientAlias PK) → allows each ingredient to store extra names

Step 7: Associative Entities (Ternary/N-ary Relationships)

- IngredientComponent (parentIngredientID PK+FK, componentIngredientID PK+FK, amount, unitID FK, formID FK) → used when one ingredient is made up of other ingredients. The fields amount, unit, and form are optional.

Final Table List:

- **Allergen**(**allergenID (PK)**, name)

- **CookingMethod**(**methodID (PK)**, name)

- **Cuisine**(**cuisineID (PK)**, name)

- **DietaryCategory**(**dietCatID (PK)**, name)

- **Ingredient**(**ingredientID (PK)**, name, **ingredientTypeID (FK)**)

- **IngredientAlias**(**ingredientID (PK)(FK)**, **alias (PK)**)

- **IngredientAllergen**(**ingredientID (PK)(FK)**, **allergenID (PK)(FK)**)
- **IngredientComponent**(**parentIngredientID (PK)(FK)**, **componentIngredientID (PK)(FK)**, amount, **unitID (FK)**, **formID (FK)**)
- **IngredientForm**(**formID (PK)**, name)
- **IngredientItem**(**recipeID (PK)(FK)**, **itemNo (PK)**, **ingredientID (FK)**, **unitID (FK)**, **formID (FK)**, amount)
- **IngredientType**(**ingredientTypeID (PK)**, name)
- **Kitchen Tool**(**toolID (PK)**, name)
- **MealPlan**(**mealplanID (PK)**, name, date, startTime, durationHours)
- **MealPlanRecipe**(**mealplanID (PK)(FK)**, **recipeID (PK)(FK)**, **role (PK)**)
- **MealType**(**mealTypeID (PK)**, name)
- **ProcessStep**(**recipeID (PK)(FK)**, **stepNo (PK)**, description, imageFilename, **methodID (FK)**)
- **Recipe**(**recipeID (PK)**, name, enteredDate, difficulty)
- **RecipeCuisine**(**recipeID (PK)(FK)**, **cuisineID (PK)(FK)**)
- **RecipeDietaryCategory**(**recipeID (PK)(FK)**, **dietCatID (PK)(FK)**)
- **RecipeMealType**(**recipeID (PK)(FK)**, **mealTypeID (PK)(FK)**)
- **StepToolUse**(**recipeID (PK)(FK)**, **stepNo (PK)(FK)**, **toolID (PK)(FK)**, usageDescription)
- **UnitOfMeasure**(**unitID (PK)**, name)

Task 3: Physical Design

Based on the logical design from Task 2, we created the physical design in MySQL Workbench. Each table includes the table name, attributes, primary key (PK), foreign keys (FK), and appropriate MySQL-compatible data types with lengths.

PHYSICAL MODEL (ERD)

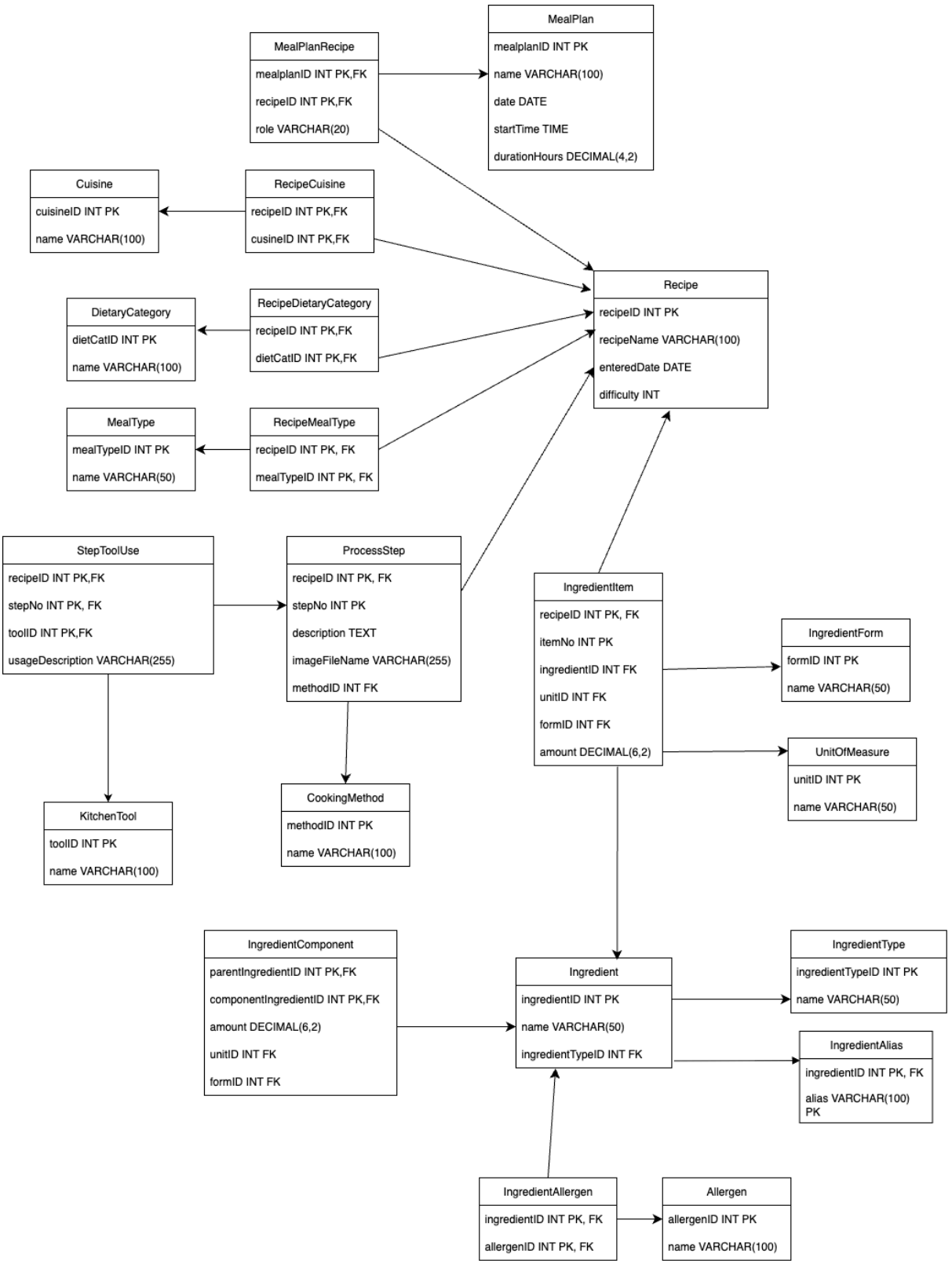
The database schema is implemented in MySQL Workbench, and the physical model is generated using DRAW.IO. The diagram below shows all entities, attributes, primary keys, foreign keys, and relationships.

Explanation

- **Keys and Relationships**: Each table contains a defined primary key, and foreign keys are used to enforce referential integrity across relationships.

- **Data Types**: (INT, VARCHAR, DATE, TIME, DECIMAL, TEXT) were used throughout the schema.

- **Data Lengths**: Reasonable lengths were assigned,(VARCHAR(50/100), DECIMAL(6,2)


MealPlan
mealplanID INT PK
name VARCHAR(100)
date DATE
startTime TIME
durationHours DECIMAL(4,2)
MealPlanRecipe
mealplanID INT PK,FK
recipeID INT PK,FK
role VARCHAR(20)
Cuisine
cuisineID INT PK
name VARCHAR(100)
RecipeCuisine
recipeID INT PK,FK
cusineID INT PK,FK
Recipe
recipeID INT PK
recipeName VARCHAR(100)
enteredDate DATE
difficulty INT
DietaryCategory
dietCatID INT PK
name VARCHAR(100)
RecipeDietaryCategory
recipeID INT PK,FK
dietCatID INT PK,FK
MealType
mealTypeID INT PK
name VARCHAR(50)
RecipeMealType
recipeID INT PK, FK
mealTypeID INT PK, FK
StepToolUse
recipeID INT PK,FK
stepNo INT PK, FK
toolID INT PK,FK
usageDescription VARCHAR(255)
ProcessStep
recipeID INT PK, FK
stepNo INT PK
description TEXT
imageFileName VARCHAR(255)
methodID INT FK
IngredientItem
recipeID INT PK, FK
itemNo INT PK
ingredientID INT FK
unitID INT FK
formID INT FK
amount DECIMAL(6,2)
IngredientForm
formID INT PK
name VARCHAR(50)
UnitOfMeasure
unitID INT PK
name VARCHAR(50)
KitchenTool
toolID INT PK
name VARCHAR(100)
CookingMethod
methodID INT PK
name VARCHAR(100)
IngredientComponent
parentIngredientID INT PK,FK
componentIngredientID INT PK,FK
amount DECIMAL(6,2)
unitID INT FK
formID INT FK
Ingredient
ingredientID INT PK
name VARCHAR(50)
ingredientTypeID INT FK
IngredientType
ingredientTypeID INT PK
name VARCHAR(50)
IngredientAlias
ingredientID INT PK, FK
alias VARCHAR(100) PK
IngredientAllergen
ingredientID INT PK, FK
allergenID INT PK, FK
Allergen
allergenID INT PK
name VARCHAR(100)